

Competitive Programming

Section 11 Editorial: Intervals And Sweep Line

Competitive Programming Course

2026-07-05

Table of contents

1	Editorial overview	4
2	A. Peak Active	5
2.1	Restatement	5
2.2	Constraints and traps	5
2.3	Observations	5
2.4	Sample walk-through	5
2.5	Algorithm	6
2.6	Correctness proof	6
2.7	Complexity	7
2.8	C++ implementation notes	7
2.9	Python implementation notes	7
2.10	Full C++ solution	7
2.11	Full Python solution	8
2.12	Common wrong answers	9
2.13	Tests that matter	9
3	B. Covered Length	10
3.1	Restatement	10
3.2	Constraints and traps	10
3.3	Observations	10
3.4	Sample walk-through	10
3.5	Algorithm	11
3.6	Correctness proof	11
3.7	Complexity	12
3.8	Full C++ solution	12
3.9	Full Python solution	13
3.10	Common wrong answers	14
3.11	Tests that matter	14
4	C. Point Coverage	15
4.1	Restatement	15
4.2	Constraints and traps	15
4.3	Observations	15
4.4	Sample walk-through	15
4.5	Algorithm	16
4.6	Correctness proof	16
4.7	Complexity	17
4.8	Full C++ solution	17
4.9	Full Python solution	18
4.10	Common wrong answers	19
4.11	Tests that matter	20
5	Stress testing sweep line	21

1 Editorial overview

This section has three sweep-line exercises.

The common idea is:

```
Convert intervals and queries into events.  
Sort events.  
Scan from left to right while maintaining active state.
```

The most important implementation detail is not sorting itself. It is deciding what happens when several events share the same coordinate.

2 A. Peak Active

2.1 Restatement

For each test case, we are given half-open intervals $[l_i, r_i)$. We need the maximum number of intervals active at the same time.

An interval is active at coordinate x when:

```
 $l_i \leq x < r_i$ 
```

2.2 Constraints and traps

The number of intervals is large, so comparing every pair is too slow.

Coordinates may be large or negative, so iterating over all time points is not valid.

The key trap is endpoint handling:

```
 $[1,2)$  and  $[2,3)$  do not overlap
```

2.3 Observations

The active count changes only at interval starts and ends.

Create two events for every interval:

```
(l, +1)  start  
(r, -1)  end
```

For half-open intervals, if a start and an end happen at the same coordinate, the end must be processed first.

With $-1 < +1$, sorting pairs (coordinate, delta) naturally gives:

```
end before start
```

2.4 Sample walk-through

Intervals:

```
[1,5), [2,6), [4,7), [7,9)
```

Events after sorting:

```
(1,+1), (2,+1), (4,+1), (5,-1), (6,-1), (7,-1), (7,+1), (9,-1)
```

Scan:

```
after 1: active = 1
after 2: active = 2
after 4: active = 3
after 5: active = 2
after 6: active = 1
after 7 end: active = 0
after 7 start: active = 1
after 9: active = 0
```

Answer:

```
3
```

2.5 Algorithm

```
events = empty
for each interval [l, r):
    add (l, +1)
    add (r, -1)

sort events by coordinate, then delta

active = 0
best = 0
for each event:
    active += delta
    best = max(best, active)

print best
```

2.6 Correctness proof

Lemma 1. Between two consecutive event coordinates, the active interval set does not change.

Proof. The only coordinates where an interval can become active or inactive are its endpoints. If an open gap contains no endpoint, no interval starts or ends inside that gap. $\square$

Lemma 2. Processing end events before start events at the same coordinate correctly models half-open intervals.

Proof. For an interval $[l, r)$, the interval is not active at coordinate r . If another interval starts at r , the first interval and second interval should not be active together because the first excludes r . Processing the end before the start removes the ending interval before the starting interval is counted. $\square$

Lemma 3. After each processed event, `active` equals the number of intervals active just after that event under the half-open convention.

Proof. Initially no processed interval is active. A start event adds exactly one interval that becomes active; an end event removes exactly one interval that stops being active. Lemma 2 handles simultaneous endpoints correctly. $\square$

Theorem. The algorithm prints the maximum active count.

Proof. By Lemma 1, the maximum can occur only immediately after processing some event coordinate. By Lemma 3, the algorithm's `active` value after each event is the correct active count there. Taking the maximum over all processed events therefore gives the true maximum. $\square$

2.7 Complexity

There are $2n$ events.

```
Time:  $O(n \log n)$ 
Memory:  $O(n)$ 
```

2.8 C++ implementation notes

Sort `pair<long long, int>`. The -1 end event naturally comes before the $+1$ start event at equal coordinates.

2.9 Python implementation notes

Python tuple sorting works the same way:

```
events.sort(key=lambda e: (e[0], e[1]))
```

2.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int solve_case(const vector<pair<long long, long long>> &intervals) {
    vector<pair<long long, int>> events;
    events.reserve(intervals.size() * 2);
    for (auto [l, r] : intervals) {
```

```

        events.push_back({l, 1});
        events.push_back({r, -1});
    }
    sort(events.begin(), events.end(), [](const auto &a, const auto &b) {
        if (a.first != b.first) return a.first < b.first;
        return a.second < b.second;
    });

    int active = 0;
    int best = 0;
    for (auto [x, delta] : events) {
        (void)x;
        active += delta;
        best = max(best, active);
    }
    return best;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<pair<long long, long long>> intervals(n);
        for (auto &[l, r] : intervals) cin >> l >> r;
        cout << solve_case(intervals) << '\n';
    }

    return 0;
}

```

2.11 Full Python solution

```

import sys

def solve_case(intervals: list[tuple[int, int]]) -> int:
    events: list[tuple[int, int]] = []
    for l, r in intervals:
        events.append((l, 1))
        events.append((r, -1))
    events.sort(key=lambda e: (e[0], e[1]))

    active = 0

```

```

best = 0
for _, delta in events:
    active += delta
    best = max(best, active)
return best

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        idx += 1
        intervals: list[tuple[int, int]] = []
        for _ in range(n):
            l = data[idx]
            r = data[idx + 1]
            idx += 2
            intervals.append((l, r))
        out.append(str(solve_case(intervals)))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

2.12 Common wrong answers

- Processing starts before ends for half-open intervals.
- Updating `best` before applying the current event.
- Treating touching intervals as overlapping.
- Iterating over all coordinates.

2.13 Tests that matter

```

[0,1), [1,2), [2,3) -> 1
[0,5), [0,5), [5,10), [5,10) -> 2
negative endpoints should behave the same way

```

3 B. Covered Length

3.1 Restatement

Given half-open intervals $[l_i, r_i)$, compute the total length covered by at least one interval.

Overlaps should be counted once.

3.2 Constraints and traps

Coordinates fit in signed 64-bit integers. The total covered length may also be large, so C++ should use `long long`.

The main trap is adding length at the wrong time.

The gap $[prev, x)$ is controlled by the active count before applying events at x .

3.3 Observations

Sort all endpoint events.

If `active > 0` from coordinate `prev` until coordinate x , then the entire gap $[prev, x)$ is covered and contributes:

```
x - prev
```

After adding that gap, apply all events at coordinate x .

Grouping same-coordinate events avoids subtle tie-order mistakes.

3.4 Sample walk-through

Intervals:

```
[1,5), [2,6), [8,10)
```

Events:

```
1:+1, 2:+1, 5:-1, 6:-1, 8:+1, 10:-1
```

Trace:

```

prev=1, active=0
x=1: add 0, active becomes 1
x=2: active>0, add 1, active becomes 2
x=5: active>0, add 3, active becomes 1
x=6: active>0, add 1, active becomes 0
x=8: active=0, add 0, active becomes 1
x=10: active>0, add 2, active becomes 0

```

Total:

7

3.5 Algorithm

```

events = all (l,+1) and (r,-1)
sort events by coordinate

active = 0
prev = first event coordinate
total = 0
i = 0

while i < number of events:
    x = events[i].coordinate
    if active > 0:
        total += x - prev
    while i < number of events and events[i].coordinate == x:
        active += events[i].delta
        i += 1
    prev = x

print total

```

3.6 Correctness proof

Lemma 1. The active count is constant on every gap between consecutive event coordinates.

Proof. No interval endpoint lies strictly inside such a gap, so no interval starts or ends inside it. $\square$

Lemma 2. If $\text{active} > 0$ before processing coordinate x , then the whole gap $[\text{prev}, x)$ is covered. If $\text{active} == 0$, none of the gap is covered.

Proof. By Lemma 1, the active count is constant throughout the gap. A positive active count means at least one interval covers every point in the gap. Zero active intervals means no interval covers any point in the gap. $\square$

Lemma 3. After processing all events at coordinate x , active is correct for the next gap.

Proof. Every interval starting at x contributes $+1$ for points after x ; every half-open interval ending at x contributes -1 because it no longer covers points after x . Applying all such deltas gives the correct active count for the next gap. $\square$

Theorem. The algorithm returns the total covered length.

Proof. The number line is partitioned into gaps between event coordinates. By Lemma 2, the algorithm adds exactly the length of each covered gap and adds nothing for uncovered gaps. By Lemma 3, the active count used for each next gap is correct. Summing over all gaps gives exactly the union length. $\square$

3.7 Complexity

Time: $O(n \log n)$
Memory: $O(n)$

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long solve_case(const vector<pair<long long, long long>> &intervals) {
    vector<pair<long long, int>> events;
    events.reserve(intervals.size() * 2);
    for (auto [l, r] : intervals) {
        events.push_back({l, 1});
        events.push_back({r, -1});
    }
    sort(events.begin(), events.end());

    long long active = 0;
    long long prev = events[0].first;
    long long total = 0;
    int i = 0;
    while (i < (int)events.size()) {
        long long x = events[i].first;
        if (active > 0) {
            total += x - prev;
        }
        while (i < (int)events.size() && events[i].first == x) {
            active += events[i].second;
            ++i;
        }
        prev = x;
    }
    return total;
}
```

```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<pair<long long, long long>> intervals(n);
        for (auto &[l, r] : intervals) cin >> l >> r;
        cout << solve_case(intervals) << '\n';
    }

    return 0;
}

```

3.9 Full Python solution

```

import sys

def solve_case(intervals: list[tuple[int, int]]) -> int:
    events: list[tuple[int, int]] = []
    for l, r in intervals:
        events.append((l, 1))
        events.append((r, -1))
    events.sort()

    active = 0
    prev = events[0][0]
    total = 0
    i = 0
    while i < len(events):
        x = events[i][0]
        if active > 0:
            total += x - prev
            while i < len(events) and events[i][0] == x:
                active += events[i][1]
                i += 1
            prev = x
    return total

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return

```

```

t = data[0]
idx = 1
out: list[str] = []
for _ in range(t):
    n = data[idx]
    idx += 1
    intervals: list[tuple[int, int]] = []
    for _ in range(n):
        l = data[idx]
        r = data[idx + 1]
        idx += 2
        intervals.append((l, r))
    out.append(str(solve_case(intervals)))
sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

3.10 Common wrong answers

- Adding $x - \text{prev}$ after applying events at x .
- Counting overlaps multiple times.
- Using `int` for length.
- Forgetting to group events with the same coordinate.

3.11 Tests that matter

```

[0,10), [0,10), [0,10) -> 10
[0,5), [5,10), [10,15) -> 15
negative intervals should contribute positive lengths

```

4 C. Point Coverage

4.1 Restatement

Given closed intervals $[l_i, r_i]$ and query points, output for each query how many intervals contain that point. Answers must be in original query order.

4.2 Constraints and traps

There can be many intervals and queries, so checking every interval for every query is too slow.

The intervals are closed. A query at an endpoint counts that interval.

The output order is the original query order, not sorted coordinate order.

4.3 Observations

Make three event types:

```
kind 0: interval starts
kind 1: query
kind 2: interval ends
```

For closed intervals, the tie order should be:

```
start before query before end
```

Then, when processing a query event, **active** equals the number of closed intervals containing that coordinate.

Store the original index of each query in its event.

4.4 Sample walk-through

Intervals:

```
[1,3], [2,5], [5,5]
```

Queries:

```
1 3 4 5 6
```

Events at coordinate 5:

```
start [5,5]
query 5
end [2,5]
end [5,5]
```

The query sees both intervals that include 5.

Answer:

```
1 2 1 2 0
```

4.5 Algorithm

```
events = empty
for each interval [l,r]:
    add (l, 0, -1)
    add (r, 2, -1)

for each query index i and coordinate x:
    add (x, 1, i)

sort events by coordinate, then kind

active = 0
answer = array of q zeros

for event in events:
    if kind == 0:
        active += 1
    else if kind == 1:
        answer[index] = active
    else:
        active -= 1

print answer
```

4.6 Correctness proof

Lemma 1. Immediately before processing a query event at coordinate x , all intervals with $l \leq x$ have already contributed their start events.

Proof. Any interval with $l < x$ has a smaller event coordinate. Any interval with $l == x$ has the same coordinate but kind 0, which sorts before query kind 1. $\square$

Lemma 2. Immediately before processing a query event at coordinate x , no interval with $r \geq x$ has been removed.

Proof. If $r > x$, the end event has a larger coordinate and has not been processed. If $r = x$, the end event has kind 2, which sorts after query kind 1. $\square$

Lemma 3. When a query at coordinate x is processed, `active` equals the number of intervals satisfying $l \leq x \leq r$.

Proof. By Lemma 1, every interval whose start is at or before x has been added. By Lemma 2, every such interval whose end is at or after x has not yet been removed. Intervals ending before x have already been removed, and intervals starting after x have not yet been added. Therefore exactly the covering intervals are active. $\square$

Theorem. The algorithm outputs the correct coverage count for every query in original order.

Proof. By Lemma 3, each query event records the correct coverage count. The event stores the query's original index, so the count is written to the correct output position. Printing the answer array therefore gives all counts in original order. $\square$

4.7 Complexity

There are $2n + q$ events.

Time: $O((n+q) \log(n+q))$
Memory: $O(n+q)$

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Event {
    long long x;
    int kind;
    int idx;
};

vector<int> solve_case(const vector<pair<long long, long long>> &intervals, const vector<long long> &queries) {
    vector<Event> events;
    events.reserve(intervals.size() * 2 + queries.size());
    for (auto [l, r] : intervals) {
        events.push_back({l, 0, -1});
        events.push_back({r, 2, -1});
    }
    for (int i = 0; i < (int)queries.size(); ++i) {
        events.push_back({queries[i], 1, i});
    }
    sort(events.begin(), events.end(), [](const Event &a, const Event &b) {
        if (a.x != b.x) return a.x < b.x;
    });
}
```

```

        return a.kind < b.kind;
    });

    int active = 0;
    vector<int> ans(queries.size());
    for (const Event &event : events) {
        if (event.kind == 0) {
            ++active;
        } else if (event.kind == 1) {
            ans[event.idx] = active;
        } else {
            --active;
        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n, q;
        cin >> n >> q;
        vector<pair<long long, long long>> intervals(n);
        for (auto &[l, r] : intervals) cin >> l >> r;
        vector<long long> queries(q);
        for (long long &x : queries) cin >> x;
        vector<int> ans = solve_case(intervals, queries);
        for (int i = 0; i < q; ++i) {
            if (i) cout << ' ';
            cout << ans[i];
        }
        cout << '\n';
    }

    return 0;
}

```

4.9 Full Python solution

```

import sys

def solve_case(intervals: list[tuple[int, int]], queries: list[int]) -> list[int]:
    events: list[tuple[int, int, int]] = []
    for l, r in intervals:

```

```

        events.append((l, 0, -1))
        events.append((r, 2, -1))
    for idx, x in enumerate(queries):
        events.append((x, 1, idx))
    events.sort()

    active = 0
    ans = [0] * len(queries)
    for _, kind, idx in events:
        if kind == 0:
            active += 1
        elif kind == 1:
            ans[idx] = active
        else:
            active -= 1
    return ans

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    t = data[0]
    idx = 1
    out: list[str] = []
    for _ in range(t):
        n = data[idx]
        q = data[idx + 1]
        idx += 2
        intervals: list[tuple[int, int]] = []
        for _ in range(n):
            l = data[idx]
            r = data[idx + 1]
            idx += 2
            intervals.append((l, r))
        queries = data[idx:idx + q]
        idx += q
        out.append(" ".join(map(str, solve_case(intervals, queries))))
    sys.stdout.write("\n".join(out))

if __name__ == "__main__":
    main()

```

4.10 Common wrong answers

- Processing end events before queries for closed intervals.
- Forgetting that $[x, x]$ contains x .
- Printing answers in sorted query order.

- Using half-open tie order on a closed-interval problem.

4.11 Tests that matter

```
[0,0], query 0 -> 1  
[0,10], [0,10], [5,5], query 5 -> 3  
duplicate query points should receive duplicate counts
```

5 Stress testing sweep line

Sweep-line bugs are often endpoint bugs. Random tests should include:

- intervals that touch exactly;
- zero-length closed intervals such as $[x, x]$;
- duplicate intervals;
- duplicate query points;
- negative coordinates;
- many events at one coordinate.

For small coordinates, brute force is simple:

Peak active:

test each event coordinate.

Covered length:

mark each integer unit $[x, x+1)$ in a small range.

Point coverage:

for each query, scan every interval directly.

The local random generators use these direct methods.

6 What to carry forward

Sweep line is not one algorithm. It is a modeling pattern.

For each new interval problem, write:

1. interval convention;
2. event types;
3. tie order;
4. active state;
5. what answer is updated at starts, ends, queries, or gaps.

Most medium contest sweep problems are solved once those five decisions are correct. Harder problems add a Fenwick tree, segment tree, heap, or set inside the sweep, but the same event discipline remains.